

Task 1:

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
GNU nano 8.7.1
int main() {
    char input[MAX_INPUT];
    char quoted_strings[MAX_QUOTES][MAX_INPUT];
    int quote_count = 0;

    printf("Session 5 - Single Quote Parsing\n");
    printf("Enter a command with single quotes:\n");
    printf("mysHELL> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    int i = 0;

    while (input[i] != '\0') {
        if (input[i] == '\') {
            i++;

            int j = 0;

            while (input[i] != '\0' && input[i] != '\') {
                if (j < MAX_INPUT - 1) {
                    quoted_strings[quote_count][j++] = input[i];
                }
                i++;
            }

            quoted_strings[quote_count][j] = '\0';

            if (input[i] == '\') {
                quote_count++;
                i++;
            } else {
                printf("Error: Unclosed single quote.\n");
                return 1;
            }
        } else {
            i++;
        }
    }

    printf("\nQuoted strings found:\n");

    if (quote_count == 0) {
        printf("No single-quoted strings found.\n");
    } else {
        for (int k = 0; k < quote_count; k++) {
            printf("Quoted String %d: [%s]\n", k + 1, quoted_strings[k]);
        }
    }

    printf("\nParsing validation:\n");

    if (quote_count > 0)
        printf("Single-quote parsing successful.\n");
    else
        printf("No quoted content to validate.\n");

    return 0;
}
```

```
ronit@Exim101:ga:~$ ./7.0.0c
Session 5 - Single Quote Parsing
Enter a command with single quotes:
myshell> 'Hello,hi'

Quoted strings found:
Quoted String 1: [Hello,hi]

Parsing validation:
Single-quote parsing successful.
```

Output:

Observation: To process single-quoted strings while preserving their contents as literal text, preventing variable expansion, storing and validating quoted strings, and handling parsing errors such as unclosed single quotes.

Task 2: To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 300
#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 100

void expand_variable(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX_INPUT - 1) {

        if (input[i] == '$') {
            i++;

            char variable[50];
            int k = 0;

            while (isalnum((unsigned char)input[i]) || input[i] == '_') {
                if (k < 49) {
                    variable[k++] = input[i];
                }
                i++;
            }

            variable[k] = '\0';

            char *value = getenv(variable);

            if (value != NULL) {
                int v = 0;

                while (value[v] != '\0' && j < MAX_INPUT - 1) {
                    output[j++] = value[v++];
                }
            }
            else {
                output[j++] = input[i++];
            }
        }

        output[j] = '\0';
    }
}

int main() {
    char input[MAX_INPUT];
    char expanded[MAX_INPUT];
    char tokens[MAX_TOKENS][MAX_TOKEN_LENGTH];
    int token_count = 0;
    int i = 0;

    printf("Session 5 - Double Quote Parsing\n");
    printf("Double quotes preserve spaces and allow variable expansion.\n");
    printf("Enter a command:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

```

GNU nano 8.7.1

```
tokens[token_count].value[1] = '\0';
strcpy(tokens[token_count].type, "DELIMITER");
token_count++;
i++;
continue;
}

while (expanded_input[i] != '\0') {
    if (expanded_input[i] == '"') {
        in_double_quote = !in_double_quote;
        i++;
        continue;
    }

    if (!in_double_quote) {
        if (isspace((unsigned char)expanded_input[i]) ||
            expanded_input[i] == ';' ||
            expanded_input[i] == '|') {
            break;
        }
    }

    if (j < MAX_TOKEN_LENGTH - 1) {
        tokens[token_count].value[j++] = expanded_input[i];
    }
    i++;
}

tokens[token_count].value[j] = '\0';

if (in_double_quote && expanded_input[i] == '\0') {
    printf("\n[ERROR] Syntax Error: Unclosed double quotes detected.\n");
    parse_error = 1;
    break;
}

if (in_double_quote) {
    strcpy(tokens[token_count].type, "QUOTED_STRING");
} else {
    strcpy(tokens[token_count].type, "ARGUMENT/COMMAND");
}

if (strlen(tokens[token_count].value) > 0 || (j == 0 && !in_double_quote)) {
    token_count++;
}
}

printf("\n===== PARSE TELEMETRY =====\n");
printf("Raw Input      : %s\n", raw_input);
printf("Expanded Input  : %s\n", expanded_input);
printf("Total Tokens    : %d\n\n", token_count);

for (int k = 0; k < token_count; k++) {
    printf(" Token [%02d] -> Type: %-16s | Value: [%s]\n", k + 1, tokens[k].type, tokens[k].value);
}
printf("===== \n");

printf("\nToken Stream Validation: ");
if (parse_error) {
    printf("FAILED\n");
    return 1;
}
}
```

26 H-1 27 H-2 28 H-3 29 H-4 30 H-5 31 H-6 32 H-7 33 H-8 34 H-9 35 H-10 36 H-11 37 H-12 38 H-13 39 H-14 40 H-15 41 H-16 42 H-17 43 H-18 44 H-19 45 H-20 46 H-21 47 H-22 48 H-23 49 H-24 50 H-25 51 H-26 52 H-27 53 H-28 54 H-29 55 H-30 56 H-31 57 H-32 58 H-33 59 H-34 60 H-35 61 H-36 62 H-37 63 H-38 64 H-39 65 H-40 66 H-41 67 H-42 68 H-43 69 H-44 70 H-45 71 H-46 72 H-47 73 H-48 74 H-49 75 H-50 76 H-51 77 H-52 78 H-53 79 H-54 80 H-55 81 H-56 82 H-57 83 H-58 84 H-59 85 H-60 86 H-61 87 H-62 88 H-63 89 H-64 90 H-65 91 H-66 92 H-67 93 H-68 94 H-69 95 H-70 96 H-71 97 H-72 98 H-73 99 H-74 100 H-75 101 H-76 102 H-77 103 H-78 104 H-79 105 H-80 106 H-81 107 H-82 108 H-83 109 H-84 110 H-85 111 H-86 112 H-87 113 H-88 114 H-89 115 H-90 116 H-91 117 H-92 118 H-93 119 H-94 120 H-95 121 H-96 122 H-97 123 H-98 124 H-99 125 H-100 126 H-101 127 H-102 128 H-103 129 H-104 130 H-105 131 H-106 132 H-107 133 H-108 134 H-109 135 H-110 136 H-111 137 H-112 138 H-113 139 H-114 140 H-115 141 H-116 142 H-117 143 H-118 144 H-119 145 H-120 146 H-121 147 H-122 148 H-123 149 H-124 150 H-125 151 H-126 152 H-127 153 H-128 154 H-129 155 H-130 156 H-131 157 H-132 158 H-133 159 H-134 160 H-135 161 H-136 162 H-137 163 H-138 164 H-139 165 H-140 166 H-141 167 H-142 168 H-143 169 H-144 170 H-145 171 H-146 172 H-147 173 H-148 174 H-149 175 H-150 176 H-151 177 H-152 178 H-153 179 H-154 180 H-155 181 H-156 182 H-157 183 H-158 184 H-159 185 H-160 186 H-161 187 H-162 188 H-163 189 H-164 190 H-165 191 H-166 192 H-167 193 H-168 194 H-169 195 H-170 196 H-171 197 H-172 198 H-173 199 H-174 200 H-175 201 H-176 202 H-177 203 H-178 204 H-179 205 H-180 206 H-181 207 H-182 208 H-183 209 H-184 210 H-185 211 H-186 212 H-187 213 H-188 214 H-189 215 H-190 216 H-191 217 H-192 218 H-193 219 H-194 220 H-195 221 H-196 222 H-197 223 H-198 224 H-199 225 H-200 226 H-201 227 H-202 228 H-203 229 H-204 230 H-205 231 H-206 232 H-207 233 H-208 234 H-209 235 H-206 236 H-207 237 H-208 238 H-209 239 H-210 240 H-211 241 H-212 242 H-213 243 H-214 244 H-215 245 H-216 246 H-217 247 H-218 248 H-219 250 H-251 252 H-253 254 H-255 256 H-257 258 H-259 260 H-261 262 H-263 264 H-265 266 H-267 268 H-269 270 H-271 272 H-273 274 H-275 276 H-277 278 H-279 280 H-281 282 H-283 284 H-285 286 H-287 288 H-289 290 H-291 292 H-293 294 H-295 296 H-297 298 H-299 300 H-301 302 H-303 304 H-305 306 H-307 308 H-309 310 H-311 312 H-313 314 H-315 316 H-317 318 H-319 320 H-321 322 H-323 324 H-325 326 H-327 328 H-329 330 H-331 332 H-333 334 H-335 336 H-337 338 H-339 340 H-341 342 H-343 344 H-345 346 H-347 348 H-349 350 H-351 352 H-353 354 H-355 356 H-357 358 H-359 360 H-361 362 H-363 364 H-365 366 H-367 368 H-369 370 H-371 372 H-373 374 H-375 376 H-377 378 H-379 380 H-381 382 H-383 384 H-385 386 H-387 388 H-389 390 H-391 392 H-393 394 H-395 396 H-397 398 H-399 400 H-401 402 H-403 404 H-405 406 H-407 408 H-409 410 H-411 412 H-413 414 H-415 416 H-417 418 H-419 420 H-421 422 H-423 424 H-425 426 H-427 428 H-429 430 H-431 432 H-433 434 H-435 436 H-437 438 H-439 440 H-441 442 H-443 444 H-445 446 H-447 448 H-449 450 H-451 452 H-453 454 H-455 456 H-457 458 H-459 460 H-461 462 H-463 464 H-465 466 H-467 468 H-469 470 H-471 472 H-473 474 H-475 476 H-477 478 H-479 480 H-481 482 H-483 484 H-485 486 H-487 488 H-489 490 H-491 492 H-493 494 H-495 496 H-497 498 H-499 500 H-501 502 H-503 504 H-505 506 H-507 508 H-509 510 H-511 512 H-513 514 H-515 516 H-517 518 H-519 520 H-521 522 H-523 524 H-525 526 H-527 528 H-529 530 H-531 532 H-533 534 H-535 536 H-537 538 H-539 540 H-541 542 H-543 544 H-545 546 H-547 548 H-549 550 H-551 552 H-553 554 H-555 556 H-557 558 H-559 560 H-561 562 H-563 564 H-565 566 H-567 568 H-569 570 H-571 572 H-573 574 H-575 576 H-577 578 H-579 580 H-581 582 H-583 584 H-585 586 H-587 588 H-589 590 H-591 592 H-593 594 H-595 596 H-597 598 H-599 600 H-601 602 H-603 604 H-605 606 H-607 608 H-609 610 H-611 612 H-613 614 H-615 616 H-617 618 H-619 620 H-621 622 H-623 624 H-625 626 H-627 628 H-629 630 H-631 632 H-633 634 H-635 636 H-637 638 H-639 640 H-641 642 H-643 644 H-645 646 H-647 648 H-649 650 H-651 652 H-653 654 H-655 656 H-657 658 H-659 660 H-661 662 H-663 664 H-665 666 H-667 668 H-669 670 H-671 672 H-673 674 H-675 676 H-677 678 H-679 680 H-681 682 H-683 684 H-685 686 H-687 688 H-689 690 H-691 692 H-693 694 H-695 696 H-697 698 H-699 700 H-701 702 H-703 704 H-705 706 H-707 708 H-709 710 H-711 712 H-713 714 H-715 716 H-717 718 H-719 720 H-721 722 H-723 724 H-725 726 H-727 728 H-729 730 H-731 732 H-733 734 H-735 736 H-737 738 H-739 740 H-741 742 H-743 744 H-745 746 H-747 748 H-749 750 H-751 752 H-753 754 H-755 756 H-757 758 H-759 760 H-761 762 H-763 764 H-765 766 H-767 768 H-769 770 H-771 772 H-773 774 H-775 776 H-777 778 H-779 780 H-781 782 H-783 784 H-785 786 H-787 788 H-789 790 H-791 792 H-793 794 H-795 796 H-797 798 H-799 800 H-801 802 H-803 804 H-805 806 H-807 808 H-809 810 H-811 812 H-813 814 H-815 816 H-817 818 H-819 820 H-821 822 H-823 824 H-825 826 H-827 828 H-829 830 H-831 832 H-833 834 H-835 836 H-837 838 H-839 840 H-841 842 H-843 844 H-845 846 H-847 848 H-849 850 H-851 852 H-853 854 H-855 856 H-857 858 H-859 860 H-861 862 H-863 864 H-865 866 H-867 868 H-869 870 H-871 872 H-873 874 H-875 876 H-877 878 H-879 880 H-881 882 H-883 884 H-885 886 H-887 888 H-889 890 H-891 892 H-893 894 H-895 896 H-897 898 H-899 900 H-901 902 H-903 904 H-905 906 H-907 908 H-909 910 H-911 912 H-913 914 H-915 916 H-917 918 H-919 920 H-921 922 H-923 924 H-925 926 H-927 928 H-929 930 H-931 932 H-933 934 H-935 936 H-937 938 H-939 940 H-941 942 H-943 944 H-945 946 H-947 948 H-949 950 H-951 952 H-953 954 H-955 956 H-957 958 H-959 960 H-961 962 H-963 964 H-965 966 H-967 968 H-969 970 H-971 972 H-973 974 H-975 976 H-977 978 H-979 980 H-981 982 H-983 984 H-985 986 H-987 988 H-989 990 H-991 992 H-993 994 H-995 996 H-997 998 H-999 1000

```

        parse_error = 1;
        break;
    }

    if (in_double_quote) {
        strcpy(tokens[token_count].type, "QUOTED_STRING");
    } else {
        strcpy(tokens[token_count].type, "ARGUMENT/COMMAND");
    }

    if (strlen(tokens[token_count].value) > 0 || (j == 0 && !in_double_quote)) {
        token_count++;
    }
}

printf("\n===== PARSE TELEMETRY =====\n");
printf("Raw Input      : %s\n", raw_input);
printf("Expanded Input  : %s\n", expanded_input);
printf("Total Tokens    : %d\n\n", token_count);

for (int k = 0; k < token_count; k++) {
    printf(" Token [%02d] -> Type: %-16s | Value: [%s]\n", k + 1, tokens[k].type, tokens[k].value);
}
printf("=====\n");

printf("\nToken Stream Validation: ");
if (parse_error) {
    printf("FAILED\n");
    return 1;
}

if (token_count == 0) {
    printf("INVALID (Empty command stream).\n");
    return 0;
}

if (strcmp(tokens[0].type, "DELIMITER") == 0) {
    printf("INVALID -> Cannot begin execution stream with delimiter '%s'.\n", tokens[0].value);
    return 0;
}

for (int k = 0; k < token_count; k++) {
    if (strcmp(tokens[k].type, "DELIMITER") == 0) {
        if (k == token_count - 1) {
            printf("INVALID -> Trailing operator '%s' at end of stream.\n", tokens[k].value);
            return 0;
        }
        if (k < token_count - 1 && strcmp(tokens[k + 1].type, "DELIMITER") == 0) {
            printf("INVALID -> Consecutive delimiters near '%s %s'.\n", tokens[k].value, tokens[k + 1].value);
            return 0;
        }
    }
}

printf("SUCCESSFULLY VALIDATED.\n");
return 0;
}

```

^{^G} Help ^{^O} Write Out ^{^F} Where Is ^{^K} Cut ^{^T} Execute ^{^C} Location ^{M-U} Undo
^{^X} Exit ^{^R} Read File ^{^_\} Replace ^{^U} Paste ^{^J} Justify ^{^/} Go To Line ^{M-E} Redo

Output:

```
Session 5 - Double Quote Parsing
Double quotes preserve spaces and allow variable expansion.
Enter a command:
myshell> echo "Hello $USER"

Tokens before variable expansion:
Token 1: [echo]
Token 2: [Hello $USER]

Variable expansion:
Token 1: [echo]
Token 2: [Hello root]

Parsing validation:
Double-quote parsing successful.
Spaces inside quotes were preserved.
Variable expansion was processed.
```

Session 6:

Task 3:

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

```

GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 300

int main() {
    char input[MAX_INPUT];
    char output[MAX_INPUT];

    int i = 0;
    int j = 0;
    int escaped = 0;

    printf("Session 6 - Escape Sequence Parsing\n");
    printf("Enter a command with escaped spaces or special characters:\n");
    printf("myshell> ");

    fgets(input, sizeof(input), stdin);
    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty input.\n");
        return 0;
    }

    while (input[i] != '\0' && j < MAX_INPUT - 1) {
        if (escaped) {
            output[j++] = input[i];
            escaped = 0;
        }
        else if (input[i] == '\\') {
            escaped = 1;
        }
        else {
            output[j++] = input[i];
        }
        i++;
    }

    if (escaped) {
        printf("\nWarning: Escape character at the end of input.\n");
    }

    output[j] = '\0';

    printf("\nOriginal input:\n");
    printf("%s\n", input);

    printf("\nProcessed input:\n");
    printf("%s\n", output);

    printf("\nParser validation:\n");
    printf("Escape sequences processed successfully.\n");
    printf("Escaped characters were preserved.\n");

    return 0;
}

```

Output:

```

Session 6 - Escape Sequence Parsing
Enter a command with escaped spaces or special characters:
myshell> hello\ world

Original input:
hello\ world

Processed input:
hello world

Parser validation:
Escape sequences processed successfully.
Escaped characters were preserved.

```

Task 4: To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

```

GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;
    int status;

    printf("Session 6 - Process Execution\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        /* Child process */
        printf("\nChild Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("Executing: ls -l\n");

        execlp("ls", "ls", "-l", NULL);

        /* This runs only if exec fails */
        perror("exec failed");
        exit(EXIT_FAILURE);
    }
    else {
        /* Parent process */
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);
        printf("Parent waiting for child...\n");

        waitpid(pid, &status, 0);
    }

    return 0;
}

```

Output:

```

Parent PID: 536
Child PID: 537
Parent waiting for child...

Child Process
Child PID: 537
Parent PID: 536
Executing: ls -l
total 332
-rw-r--r-- 1 root root 53 Aug 8 06:04 destination.txt
drwxr-xr-x 5 root root 4096 Aug 11 06:52 ossp_2520090049
-rw-r--r-- 1 root root 41 Aug 8 08:32 sample.txt
-rwxr-xr-x 1 root root 16232 Aug 11 10:12 session2_task1
-rw-r--r-- 1 root root 559 Aug 11 10:15 session2_task1.c
-rwxr-xr-x 1 root root 16104 Aug 11 14:39 session2_task2
-rw-r--r-- 1 root root 537 Aug 11 14:39 session2_task2.c
-rwxr-xr-x 1 root root 16680 Aug 11 16:22 session3_task1
-rw-r--r-- 1 root root 4230 Aug 11 16:22 session3_task1.c
-rwxr-xr-x 1 root root 16400 Aug 11 16:34 session3_task2
-rw-r--r-- 1 root root 2188 Aug 11 16:34 session3_task2.c
-rwxr-xr-x 1 root root 16192 Aug 11 17:28 session4_task1
-rw-r--r-- 1 root root 2266 Aug 11 17:20 session4_task1.c
-rwxr-xr-x 1 root root 16280 Aug 11 17:41 session4_task2
-rw-r--r-- 1 root root 2252 Aug 11 17:40 session4_task2.c
-rwxr-xr-x 1 root root 16192 Aug 12 03:09 session5_task1
-rw-r--r-- 1 root root 1552 Aug 12 03:09 session5_task1.c
-rwxr-xr-x 1 root root 16320 Aug 12 11:44 session5_task2
-rw-r--r-- 1 root root 3236 Aug 12 11:53 session5_task2.c
-rwxr-xr-x 1 root root 16192 Aug 12 11:58 session6_task1
-rw-r--r-- 1 root root 1222 Aug 12 11:58 session6_task1.c
-rwxr-xr-x 1 root root 16360 Aug 12 12:02 session6_task2
-rw-r--r-- 1 root root 1190 Aug 12 12:02 session6_task2.c
-rwxr-xr-x 1 root root 16312 Aug 11 07:10 skill_week1_task2
-rw-r--r-- 1 root root 879 Aug 11 09:58 skill_week1_task2.
-rw-r--r-- 1 root root 53 Aug 8 06:03 source.txt
-rwxr-xr-x 1 root root 16528 Aug 8 04:41 week1pt1
-rw-r--r-- 1 root root 1350 Aug 8 04:41 week1pt1.c
-rwxr-xr-x 1 root root 16224 Aug 8 06:03 week2pt1
-rw-r--r-- 1 root root 1208 Aug 8 06:02 week2pt1.c
-rwxr-xr-x 1 root root 16304 Aug 8 03:26 week3pt1
-rw-r--r-- 1 root root 1023 Aug 8 03:26 week3pt1.c
-rwxr-xr-x 1 root root 16136 Aug 8 03:33 week3pt2
-rw-r--r-- 1 root root 376 Aug 8 03:32 week3pt2.c

Child exited with status: 0
Parent process completed.

```

Observation:

child process using `fork()`, execute another program with `exec()`, pass arguments to the new program, and use `waitpid()` to synchronize the parent with the child